

AIGC检测 · 全文报告单

NO:CNKIAIGC2026FJ_202605130126690

检测时间: 2026-05-14 16:43:08

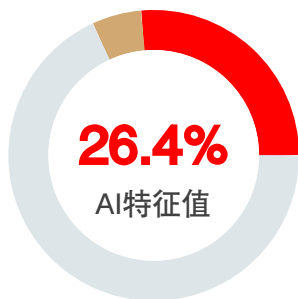
篇名: 基于CNN与YuNet的人脸表情识别系统设计与实现

作者: 张吉鹏

单位:

文件名: 1356339_张吉鹏_基于CNN与YuNet的人脸表情识别系统设计与实现.docx

全文检测结果



AI特征值: 26.4%

AI特征字符数: 8615

总字符数: 32580

- AI特征显著 (计入AI特征字符数)
- AI特征疑似 (未计入AI特征字符数)
- 未标识部分

AIGC片段分布图

前部20%

AI特征值: 6.2%

AI特征字符数: 405

中部60%

AI特征值: 37.0%

AI特征字符数: 7229

后部20%

AI特征值: 15.1%

AI特征字符数: 981



分段检测结果

序号	AI特征值	AI特征字符数 / 章节(部分)字符数	章节(部分)名称
1	11.1%	405 / 3645	中英文摘要等
2	12.7%	537 / 4243	第1章绪论
3	87.4%	5285 / 6050	第2章系统需求分析与架构设计
4	16.6%	1025 / 6189	第3章表情识别模型构建与训练

5	9.7%	382 / 3955	第4章模型训练优化
6	10.7%	670 / 6273	第5章系统实现与运行测试
7	14.0%	311 / 2225	第6章结论与展望

1. 中英文摘要等			
AI特征值: 11.1%		AI特征字符数 / 章节(部分)字符数: 405 / 3645	

片段指标列表

序号	片段名称	字符数	AI特征		
1	片段1	405	显著		11.1%

原文内容

摘要

本文围绕树莓派端实时人脸表情识别任务，完成了一套基于CNN与YuNet的人脸表情识别系统。系统以摄像头实时画面为输入，先利用YuNet检测人脸位置和关键点，再将裁剪后的人脸区域输入MobileNetV3表情分类模型，最终在显示界面中输出人脸框、表情类别、置信度和运行状态信息。与只进行离线图像分类的实验不同，本文更关注模型训练、格式转换和端侧运行之间的衔接过程。

在表情分类模型训练方面，本文以FER2013格式数据为基础，对样本进行了扩充、合并、清洗和重新划分，构建训练集、验证集、测试集和无标签集。模型采用MobileNetV3-large作为骨干网络，并将最终分类层调整为七类表情输出，用于识别愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。训练过程中结合数据增强、类别均衡、标签平滑、学习率调度和伪标签辅助训练等方法，以提高模型在类别不均衡条件下的训练稳定性。

在系统实现方面，本文完成了PyTorch模型训练、ONNX模型导出、TensorFlow SavedModel转换和TensorFlow Lite模型生成，并将FP16量化后的TFLite模型部署到Raspberry Pi 5平台。树莓派端程序集成了摄像头读取、YuNet人脸检测、人脸区域裁剪、TFLite表情分类、结果绘制和图像保存等功能。为了降低端侧运行负载，程序采用缩小检测输入尺寸、间隔执行检测、限制单帧分类人脸数量和异步读取摄像头画面等方式提高运行连续性。

实验结果表明，最终Stage 3表情分类模型在测试集上取得90.77%的准确率和87.43%的宏平均F1值。树莓派端运行测试表明，系统能够在室内近距离场景下完成

实时人脸检测与表情识别，运行画面较为连续，检测框、关键点、类别和置信度能够正常显示，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。结果说明，本文系统实现了从模型训练到端侧部署运行的完整流程，具备一定的实时性、稳定性和工程应用价值。

关键词：人脸表情识别；卷积神经网络；YuNet；MobileNetV3；树莓派；端侧部署

ABSTRACT

This paper designs and implements a facial expression recognition system based on CNN and YuNet for real-time edge deployment on a Raspberry Pi. The system takes camera frames as input, detects facial regions and landmarks using YuNet, and then feeds the cropped face region into a MobileNetV3-based expression classification model. The recognition result, including the face bounding box, expression category, confidence score, and runtime information, is displayed on the screen. Compared with an offline image classification experiment, this work focuses more on the complete process from model training and model conversion to edge-side real-time operation.

For expression classification, datasets in the FER2013 format are used as the basis. The samples are expanded, merged, cleaned, and re-split into training, validation, test, and unlabeled sets. MobileNetV3-large is adopted as the backbone network, and its final classification layer is modified to output seven expression categories, including anger, disgust, fear, happiness, sadness, surprise, and neutrality. During training, data augmentation, class balancing, label smoothing, learning rate scheduling, and pseudo-label-assisted training are used to improve training stability under imbalanced class distribution.

For system implementation, the trained PyTorch model is exported to ONNX, converted to TensorFlow SavedModel, and finally converted to TensorFlow Lite. The FP16-quantized TFLite model is deployed on the Raspberry Pi 5 platform. The edge-side program integrates camera reading, YuNet-based face detection, face cropping, TFLite expression inference, result visualization, and image saving. To reduce the computational load on the edge device, the program uses a smaller detection input size, interval-based face detection, limited per-frame face classification, and asynchronous camera reading.

Experimental results show that the final Stage 3 expression

classification model achieves an accuracy of 90.77% and a macro-average F1 score of 87.43% on the test set. Runtime tests on the Raspberry Pi show that the system can perform real-time face detection and facial expression recognition in indoor close-range scenarios. The video display remains relatively smooth, and the bounding box, landmarks, expression category, and confidence score can be updated normally.

No program crash, camera interruption, or interface freeze is observed during continuous operation. These results indicate that the system completes the full workflow from model training to edge deployment and has practical real-time performance, stability, and engineering value.

Key Words: facial expression recognition; convolutional neural network; YuNet; MobileNetV3; Raspberry Pi; edge deployment

11.1%(405)

2. 第1章绪论

AI特征值: 12.7% AI特征字符数 / 章节(部分)字符数: 537 / 4243

片段指标列表

序号	片段名称	字符数	AI特征		
2	片段1	537	显著		12.7%

原文内容

第1章 绪论

1.1 研究背景与意义

随着人工智能、计算机视觉和嵌入式计算技术的发展，智能系统的功能已经从简单的信息处理逐渐扩展到对用户状态的感知与反馈。人脸表情是人类表达情绪的重要方式之一，具有直观性强、采集方便和非接触等特点，因此人脸表情识别逐渐成为情感计算和人机交互领域中的重要研究方向[1-3]。近年来，人脸表情识别已被应用于智能交互、课堂状态分析、驾驶员状态监测、心理健康辅助和服务机器人等场景，其研究价值不仅体现在识别算法本身，也体现在实际系统的部署和运行能力上。

人脸表情识别的基本任务是从图像或视频中获取人脸区域，并对其情绪类别进行判断。常见表情类别通常包括愤怒、厌恶、恐惧、高兴、中性、悲伤和惊讶等。FER2013等公开数据集为表情识别模型训练和方法对比提供了基础数据来源[4]，使

不同模型能够在较统一的数据标准下进行测试与评价。随着深度学习的发展，卷积神经网络逐渐取代传统人工特征方法，成为人脸表情识别中的主流技术路线。相比传统纹理特征和浅层分类器，卷积神经网络能够从图像中自动提取多层次特征，在复杂表情图像识别中具有更强的表达能力。

但是，人脸表情识别系统在实际应用中并不只需要关注离线测试准确率。对于树莓派、移动终端和边缘计算设备等资源受限平台而言，模型的参数量、计算复杂度、推理速度和运行稳定性同样重要[12-15]。较复杂的深度模型虽然可能在数据集上取得较高准确率，但在端侧平台上运行时容易出现推理延迟高、画面卡顿、内存占用大和连续运行不稳定等问题。因此，如何在保证基本识别效果的前提下，降低模型计算量并完成端侧部署，是当前人脸表情识别系统设计中需要重点解决的问题[1][12][13]。

在轻量化表情识别研究中，研究者通常从网络结构优化、注意力机制、特征融合和模型压缩等角度提升模型效率。例如，基于轻量化ResNet、MobileNet和Transformer改进的表情识别方法，均尝试在参数量、计算量和识别准确率之间取得平衡[5-8]。其中，MobileNetV3由于具有较好的移动端适应性，适合作为嵌入式视觉任务中的轻量化骨干网络[9]。在人脸检测环节，YuNet作为面向边缘设备设计的轻量级人脸检测模型，能够在较低计算开销下完成人脸定位和关键点检测，为端侧实时表情识别提供了可行的前端检测方案[10]。

基于上述背景，本毕业设计围绕端侧人脸表情识别系统的设计与实现展开研究。系统采用“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”的整体方案，前端利用YuNet完成人脸检测与关键点定位，后端基于MobileNetV3构建七类表情分类模型，并将训练后的模型部署到树莓派平台，实现从视频采集、人脸检测、表情分类到结果显示的完整流程。相比单纯进行离线模型训练，本设计更强调模型训练、模型转换、端侧部署和运行测试之间的闭环实现，具有较强的工程实践意义。

1.2 国内外研究现状

人脸表情识别技术经历了由传统人工特征方法向深度学习方法发展的过程。早期方法通常先进行人脸检测和预处理，再提取局部纹理、几何结构或统计特征，最后结合分类器完成表情判断。这类方法结构较清晰，计算量相对可控，但对光照变化、姿态偏转、遮挡和个体差异较敏感，在复杂自然场景下识别稳定性有限。近年来的综述研究表明，深度学习已经成为人脸表情识别领域的主流方向，研究重点也逐渐从单一模型准确率扩展到数据质量、模型泛化能力、轻量化设计和实际部署等方面[1-3]。

在深度学习方法中，卷积神经网络能够直接从表情图像中学习特征，减少人工设计特征对经验的依赖。FER2013数据集的提出推动了深度表情识别方法的发展，使模型可以围绕七类基本表情进行训练和测试[4]。此后，研究者通过改进卷积网络结构、引入注意力机制、融合多尺度特征等方式提高表情识别效果。赵晓等提出基于

注意力机制的ResNet轻量网络，通过分组卷积、倒残差结构和多尺度注意力模块减少模型参数量，并在FER2013和CK+数据集上进行了验证[5]。这类研究说明，轻量化设计不仅可以降低模型复杂度，也可以通过合理的特征增强方法保持模型的识别能力。

MobileNet系列网络是轻量化卷积神经网络中的代表结构之一。左义海等提出NCA-MobileNet方法，通过改进MobileNetV3结构并引入注意力机制，实现了面向人脸表情识别任务的轻量化优化[6]。李艳秋等将轻量型Swin Transformer与多尺度特征融合模块相结合，用于改善模型在表情细节捕捉和实时性方面的表现[7]。Jiang等则围绕改进MobileNetV3的人脸表情识别算法展开研究，说明MobileNetV3在表情识别任务中仍具有进一步优化和应用价值[8]。MobileNetV3原始模型通过硬件感知网络搜索和结构优化，在移动端视觉任务中实现了较好的精度与效率平衡[9]。因此，本文选用MobileNetV3作为表情分类模型的骨干网络，能够较好地契合树莓派端部署对模型规模和推理效率的要求。

在人脸检测方面，检测结果的准确性会直接影响后续表情分类输入质量。传统或较复杂的人脸检测模型虽然能够取得较高检测精度，但在端侧平台上运行时可能带来较高计算开销。YuNet是一种面向边缘设备设计的轻量级人脸检测模型，具有模型小、速度快和便于集成等特点，适合作为实时表情识别系统的前端检测模块[10]。此外，也有研究通过改进S3FD等检测网络提高复杂场景下的人脸检测效果[11]。对于本文所设计的系统而言，检测模块不只需要找到人脸，还需要保证在视频流中连续运行，因此轻量化检测方案更符合树莓派端应用需求。

随着边缘计算和嵌入式AI的发展，人脸表情识别系统的研究逐渐从PC端离线测试转向端侧实时部署。Mohammadi等针对边缘设备上的人脸表情识别进行了实验，比较了CPU、GPU、VPU和TPU等不同硬件平台下的运行表现，说明FER模型的实际应用效果与硬件平台、模型结构和推理优化方式密切相关[12]。Liu等对边缘设备上的高效神经网络进行了综述，指出量化、剪枝和网络结构设计是提高端侧模型运行效率的重要方向[13]。Weng等对神经网络量化方法进行了总结，认为量化可以通过降低数值精度来减少模型复杂度和推理开销[14]。Wei等进一步综述了神经网络量化技术的发展，说明模型压缩和低精度推理仍是端侧部署中的重要研究内容[15]。

综合国内外研究可以看出，人脸表情识别已经形成较为完整的发展路径：传统方法侧重人工特征提取，深度学习方法提升了模型的特征表达能力，轻量化网络和边缘部署技术则推动表情识别系统从离线实验走向实际应用。但在树莓派等资源受限平台上同时完成人脸检测、表情分类、结果显示和稳定运行，仍需要对模型选型、训练过程、模型转换和端侧推理流程进行综合设计。基于此，本文将研究重点放在YuNet人脸检测、MobileNetV3表情分类和树莓派端部署测试上，力求完成一套能够实际运行的端侧人脸表情识别系统。

1.3 本文主要研究内容

围绕端侧人脸表情识别系统的设计与实现，本文主要开展以下几方面工作。

第一，完成系统需求分析与总体架构设计。结合树莓派端实时视觉识别场景，明确系统需要完成摄像头视频采集、人脸检测、人脸区域裁剪、表情分类、结果显示和图像保存等功能。在此基础上，设计系统整体流程，将系统划分为图像采集模块、人脸检测模块、图像预处理模块、表情分类模块、结果显示模块和数据保存模块，为后续模型训练和系统实现提供结构基础。

第二，完成基于MobileNetV3的表情分类模型构建。以FER2013格式数据为训练和测试基础，对图像数据进行尺寸调整、归一化和数据增强处理。模型以MobileNetV3为骨干网络，并将最终分类层调整为七类表情输出。训练过程中结合类别均衡、标签平滑、学习率调度和早停机制等方法提升训练稳定性。对于伪标签相关方法，本文将其作为训练优化策略使用，不将其作为核心创新点进行强调。

第三，完成YuNet人脸检测与表情分类模型的系统集成。系统前端采用YuNet对摄像头输入图像进行人脸检测，并根据检测框裁剪人脸区域；后端将裁剪后的人脸图像输入MobileNetV3表情分类模型，得到对应表情类别和置信度结果。通过检测与分类模块的组合，实现从视频帧输入到表情识别结果输出的完整处理流程。

第四，完成模型转换与树莓派端部署。训练完成后，将PyTorch模型转换为适合端侧运行的格式，并在树莓派平台上完成推理程序设计。针对端侧平台算力有限的问题，系统在实际部署时采用降低检测输入分辨率、间隔执行人脸检测、限制单次分类人脸数量和异步读取视频帧等方式减少运行负载，提高系统实时性和连续运行能力。

第五，完成模型评价与系统运行测试。通过测试集准确率、宏平均F1值和混淆矩阵等指标评价表情分类模型性能；通过功能测试、实时性测试和稳定性测试评价树莓派端系统运行效果。最终从模型识别效果和端侧运行效果两个角度，验证本文系统是否完成从训练到部署的完整闭环。

1.4 本文结构安排

本文共分为六章，各章内容安排如下。

第1章为绪论。主要介绍人脸表情识别的研究背景与意义，梳理传统方法、深度学习方法、轻量化网络和端侧部署相关研究现状，并说明本文的主要研究内容和结构安排。

第2章为系统需求分析与架构设计。主要分析端侧人脸表情识别系统的应用场景、功能需求和性能需求，设计系统总体架构，并对图像采集、人脸检测、表情分类和结果输出等模块进行划分。

第3章为表情识别模型构建与训练。主要介绍数据集构建与预处理方法，说明MobileNetV3表情分类网络的结构调整过程，并对数据增强、类别均衡和训练配置等内容进行说明。

第4章为模型训练优化与实验结果分析。主要围绕模型训练过程展开，分析基础

12.7%(537)

监督训练、多阶段训练和伪标签辅助优化对模型效果的影响，并通过准确率、宏平均F1值和混淆矩阵等指标评价最终模型性能。

第5章为系统实现、模型部署与运行测试。主要介绍模型导出与转换流程、树莓派端推理程序设计、摄像头实时识别流程集成，以及系统功能、实时性和稳定性测试结果，重点验证系统在端侧平台上的实际运行效果。

第6章为结论与展望。主要总结本文完成的工作，分析系统当前存在的不足，并从模型优化、多场景适应性和端侧部署性能等方面提出后续改进方向。

3. 第2章系统需求分析与架构设计

AI特征值: 87.4%

AI特征字符数 / 章节(部分)字符数: 5285 / 6050

片段指标列表

序号	片段名称	字符数	AI特征		
3	片段1	1541	显著		25.5%
4	片段2	2955	显著		48.8%
5	片段3	789	显著		13.0%

原文内容

第2章系统需求分析与架构设计

2.1 系统需求分析

本毕业设计面向端侧人脸表情识别场景，设计并实现一套能够在树莓派平台上运行的实时识别系统。系统以摄像头采集到的视频画面作为输入，先通过人脸检测模型定位画面中的人脸区域，再将裁剪后的人脸图像送入表情分类模型，最后在显示界面中给出人脸框、表情类别和置信度等结果。

与单纯的离线图像分类实验相比，本文系统需要同时考虑模型训练效果和端侧运行效果。训练阶段主要在具备RTX3060 GPU的PC机上完成，用于完成数据处理、模型训练、模型评估和模型导出；部署阶段则以Raspberry Pi 5为运行平台，配合摄像头完成实时检测与表情识别。因此，系统设计既要保证表情分类模型具有一定识别能力，也要保证模型能够在树莓派端完成连续推理。

结合课题目标和实际实现条件，本文从应用场景、功能需求、性能需求和部署需求四个方面进行分析。应用场景用于限定系统主要使用范围，功能需求用于明确系统需要完成的处理流程，性能需求用于约束实时性和稳定性，部署需求则用于保证训练模型能够转换并运行在端侧设备上。

2.1.1 应用场景与任务目标

本文系统主要面向室内近距离人机交互场景，例如课堂状态观察、智能终端交互演示和情绪识别原型系统验证等。在这类场景中，摄像头通常固定在桌面、显示器或嵌入式设备附近，用户与摄像头之间保持相对稳定的距离，环境光照和背景变化也相对有限。基于这一使用条件，本文不将远距离监控、多角度遮挡和复杂多人场景作为主要研究对象，而是重点验证单人或少量人脸情况下的端侧实时识别流程。

系统识别对象为人脸图像中的七类基本表情，包括愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。系统运行时，摄像头连续采集图像帧，YuNet模型从图像中检测人脸位置，程序根据检测框截取人脸区域，并将其调整为分类模型所需的输入尺寸。随后，MobileNetV3表情分类模型输出各类别概率，系统选取概率最高的类别作为当前表情识别结果。

从任务目标上看，本文并不以提出新的复杂网络结构为重点，而是围绕“模型训练—模型转换—树莓派部署—实时测试”这一完整流程展开。也就是说，本文不仅关注模型在测试集上的准确率，还关注模型能否在树莓派端稳定运行，并通过摄像头画面直观展示识别结果。

2.1.2 功能需求分析

根据系统任务目标，本文系统主要包括以下功能。

第一，视频采集功能。系统需要调用摄像头获取实时画面，并将读取到的图像帧送入后续处理流程。由于实时识别对画面连续性要求较高，摄像头读取过程不能长时间阻塞主循环，否则会造成显示延迟或画面卡顿。因此，视频采集模块需要保证输入画面稳定，并尽量减少对检测和分类过程的影响。

第二，人脸检测功能。系统需要在视频帧中定位人脸区域，并输出人脸检测框和关键点信息。本文选用YuNet作为前端检测模型，主要原因是该模型体积较小、调用方便，适合嵌入式端侧系统集成。检测结果直接决定后续裁剪区域的质量，如果人脸框偏移较大，分类模型接收到的图像可能缺少有效表情信息，因此检测模块需要兼顾速度和定位稳定性。

第三，图像预处理功能。系统根据检测框裁剪人脸区域后，需要对图像进行尺寸缩放、颜色通道转换和归一化处理，使端侧输入格式与训练阶段保持一致。本文表情分类模型输入尺寸为 224×224 ，训练和推理阶段均采用相同的归一化方式，以减少模型转换后输入分布不一致带来的影响。

第四，表情分类功能。系统将预处理后的人脸图像输入MobileNetV3分类模型，得到七类表情的预测概率。程序根据最大概率确定表情类别，并结合置信度显示识别结果。考虑到实际摄像头画面中可能存在表情不明显、姿态变化或光照不均等情况，系统输出结果时保留置信度信息，便于观察模型判断是否稳定。

第五，结果显示功能。系统需要在实时画面中绘制人脸框，并在框附近显示表

25.5%(1541)

情类别和置信度。同时，为了便于调试和测试，界面中还可以显示帧率、检测耗时和分类耗时等信息。这样既能展示识别效果，也能辅助判断系统运行瓶颈。

第六，结果保存与调试功能。系统应能够保存部分识别画面或测试结果，用于后续分析和毕业设计展示。该功能不属于识别流程的核心环节，但有助于记录不同表情类别下的运行效果，也方便在系统测试中检查误识别情况。

2.1.3 性能需求分析

端侧人脸表情识别系统除了功能完整外，还需要满足一定的性能要求。由于树莓派平台的计算资源有限，系统设计不能只追求离线测试准确率，还需要综合考虑模型规模、推理速度和连续运行稳定性。

首先，系统需要具备基本识别准确率。表情分类模型应能够在测试集上取得较稳定的准确率和宏平均F1值，并且在摄像头实际画面中能够对常见表情作出合理判断。由于不同表情之间存在相似性，部分样本还会受到光照、姿态和个体差异影响，因此模型评价时不能只看整体准确率，还需要结合混淆矩阵和各类别表现进行分析。

其次，系统需要满足实时运行要求。实时性主要体现在画面是否连续、检测框是否能及时更新、表情结果是否能够随人脸变化而变化。树莓派端同时运行人脸检测和表情分类时，如果每一帧都完整执行检测与分类，容易增加计算负担。为此，系统在设计上采用轻量化检测模型和轻量化分类模型，并在部署阶段通过检测间隔、分类间隔和单帧分类人脸数限制来降低负载。

再次，系统需要控制资源占用。表情分类模型需要在端侧设备上完成推理，因此模型结构不宜过大。本文选用MobileNetV3作为分类骨干，并在部署阶段将模型转换为TensorFlow Lite格式，以便在树莓派端加载运行。与直接使用PC端训练模型相比，转换后的端侧模型更适合嵌入式运行环境。

最后，系统需要具备连续运行稳定性。系统测试过程中应能够持续完成摄像头读取、人脸检测、表情分类和结果显示，避免出现摄像头中断、界面卡死或程序异常退出等问题。稳定性不仅取决于模型大小，也与摄像头读取方式、检测频率、推理线程数和结果显示流程有关。因此，本文在系统实现与测试阶段对这些因素进行综合调整。

2.2 系统总体架构设计

2.2.1 系统总体架构概述

根据上述需求，本文采用分层设计思路构建端侧人脸表情识别系统。系统整体技术路线如图2-1所示，主要包括硬件层、数据层、模型训练与伪标签生成层、模型评估与分析层以及在线推理与应用层。各层按照“数据准备—模型训练—模型评价—模型转换—端侧运行”的顺序衔接，最终形成从离线训练到树莓派实时识别的完整流程。

图 2-1 端侧人脸表情识别系统总体技术路线图

48.8%(2955)

硬件层主要包括PC训练平台、Raspberry Pi 5和摄像头设备。PC端用于完成数据整理、模型训练和模型导出；树莓派端用于加载转换后的模型，并运行摄像头实时识别程序；摄像头负责采集输入画面，为系统提供连续图像帧。

数据层主要包括有标签数据、无标签数据和伪标签数据。有标签数据用于基础监督训练和模型评估，无标签数据用于伪标签生成，筛选后的伪标签数据则作为后续训练优化的补充样本。通过这种方式，系统能够在已有标注数据基础上进一步利用无标签样本，提高训练数据的利用率。

模型训练与伪标签生成层包括基础监督训练、伪标签生成和多阶段训练三个部分。首先，系统利用有标签样本训练初始分类模型；然后使用训练后的模型对无标签数据进行推理，筛选置信度较高的样本生成伪标签；最后，将真实标注样本与伪标签样本结合，用于后续模型优化。

模型评估与分析层主要负责验证模型效果。本文通过准确率、宏平均F1值和混淆矩阵等指标评价模型性能，判断模型是否满足部署要求。该部分结果也为第4章的训练优化分析提供依据。

在线推理与应用层是系统最终运行部分。摄像头采集图像后，系统先调用YuNet进行人脸检测，再对人脸区域进行裁剪和预处理，随后输入MobileNetV3表情分类模型，最后将识别结果绘制在实时画面中。该层体现了本文从算法训练到端侧应用的工程闭环。

2.2.2 系统识别流程设计

为了说明树莓派端程序的运行过程，本文对实时识别流程进行设计，如图2-2所示。该流程主要包括系统初始化、视频帧读取、人脸检测、人脸裁剪与预处理、表情分类、结果显示与保存等步骤。

图 2-2 系统识别流程图

系统启动后，首先加载YuNet人脸检测模型和TensorFlow Lite表情分类模型，同时初始化摄像头、类别标签和显示窗口。如果摄像头无法正常打开，程序需要及时提示异常，避免进入无效循环。

完成初始化后，系统从摄像头读取当前帧。对于读取成功的图像，程序先进行必要的尺寸处理，再送入人脸检测模块。若当前帧未检测到人脸，则直接显示画面并等待下一帧；若检测到人脸，则根据检测框进入后续处理流程。

在人脸裁剪阶段，程序会对检测框进行边界检查，并适当扩展人脸区域，使裁剪图像尽量包含完整面部信息。裁剪后的人脸图像被缩放到 224×224 ，并完成通道转换和归一化处理。该步骤需要与训练阶段保持一致，否则可能影响分类模型输出。

表情分类阶段将预处理后的人脸图像输入TFLite模型，得到七类表情概率。系统取最大概率对应类别作为识别结果，并记录该类别的置信度。最后，程序在实时画面中绘制人脸框、关键点、表情类别和置信度，同时根据设定条件保存部分识别

画面，用于后续实验分析和效果展示。

2.2.3 系统功能模块划分

结合系统架构和识别流程，本文将端侧人脸表情识别系统划分为五个功能模块。

1. 图像采集模块

图像采集模块负责从摄像头读取实时视频帧，是系统输入端。该模块需要保证画面读取连续，并将图像传递给后续检测流程。在树莓派端运行时，摄像头读取速度会影响整体流畅度，因此程序设计时需要尽量减少帧读取阻塞。

2. 人脸检测模块

人脸检测模块负责在视频帧中定位人脸区域。本文采用 YuNet 输出人脸检测框和关键点信息，为后续人脸裁剪提供依据。该模块既要保证检测结果可用，也要控制检测耗时，避免影响实时显示。

3. 图像预处理模块

图像预处理模块负责将检测到的人脸区域转换为分类模型可接受的输入格式，主要包括人脸裁剪、边界修正、尺寸缩放、颜色转换和归一化处理。预处理结果需要与训练阶段输入保持一致，这是保证模型部署效果的重要条件。

4. 表情识别模块

表情识别模块负责对人脸图像进行七类表情分类。本文使用 MobileNetV3 作为分类骨干网络，并将最终输出层调整为七类表情。该模块输出表情类别和预测置信度，是系统识别结果的主要来源。

5. 结果输出与管理模块

结果输出与管理模块负责将识别结果展示给用户，包括绘制人脸框、关键点、表情类别、置信度和运行状态信息。同时，该模块根据需要保存部分识别画面，便于后续整理实验结果和展示系统运行效果。

2.3 关键技术与方法选型

本节对系统中使用的关键技术进行选型说明。第2章只从系统设计角度说明技术选择理由，具体模型结构、训练参数、实验结果和部署细节将在第3章至第5章中进一步展开。

2.3.1 人脸检测模型选型

人脸检测是表情识别系统的前置环节，其检测结果会直接影响后续分类效果。如果检测框偏移较大，人脸区域可能缺少关键表情信息；如果检测耗时过长，系统整体帧率会下降。因此，检测模型需要在精度、速度和部署便利性之间取得平衡。

本文选用YuNet作为人脸检测模型。YuNet是一种轻量级人脸检测模型，能够输出人脸框和关键点信息，如图2-3所示，适合在边缘设备或实时应用中使用。与较复杂的人脸检测网络相比，YuNet更便于在OpenCV环境下调用，也更符合树莓派端系统对轻量化和实时性的要求[10]。

在本文系统中，YuNet主要用于摄像头画面中的人脸定位。为了减少树莓派端的计算压力，部署阶段可通过缩小检测输入尺寸、设置检测间隔等方式降低检测负载。这样既能维持检测结果的连续性，又能避免检测模块占用过多时间。

图 2-3 YuNet人脸检测示意图

2.3.2 表情分类模型选型

表情分类模型需要对裁剪后的人脸图像进行七类表情判断。考虑到系统最终部署在树莓派端，分类模型不能过于复杂，否则会增加推理延迟和内存占用。与此同时，模型仍需要具备一定的特征提取能力，以区分相似表情类别。

MobileNetV3是一种面向移动端和资源受限设备设计的轻量化卷积神经网络。该网络通过深度可分离卷积、倒残差结构和轻量化注意力模块减少计算量，同时保持较好的特征表达能力[9]，如图2-4所示。因此，本文选用MobileNetV3作为表情分类模型骨干网络，并将最终分类层调整为七类输出。

在本文任务中，输入人脸图像统一调整为224×224。由于表情图像中五官局部变化对分类结果影响较大，模型需要从眼部、嘴部和面部纹理等区域提取有效特征。MobileNetV3在模型规模和推理效率方面较适合端侧部署，因此能够满足本文对实时性和工程可实现性的要求。

图 2-4 MobileNetV3网络结构示意图

2.3.3 训练优化方法选型

表情识别模型的效果不仅取决于网络结构，也与数据质量和训练策略有关。FER2013 类数据中不同类别样本数量存在差异，部分类别之间表情边界也较模糊，容易造成模型对少数类识别不足或对相似类别产生混淆。因此，训练阶段需要采用一定的优化方法提升模型稳定性。

本文训练阶段主要采用数据增强、类别均衡、伪标签筛选和多阶段训练等策略。数据增强用于增加样本变化，提高模型对姿态、光照和局部扰动的适应能力；类别均衡用于减弱样本数量差异对训练过程的影响；伪标签方法则用于利用无标签样本，为后续训练提供补充监督信息。

需要说明的是，本文并不将伪标签方法作为核心理论创新点，而是将其作为训练优化手段。系统首先利用有标签数据训练基础模型，再通过模型对无标签数据进行预测，筛选高置信度样本生成伪标签，最后将伪标签样本与真实标注样本结合进行后续训练。该流程有助于扩充训练数据，但其效果仍需要通过验证集和测试集结果进行判断。

2.4 本章小结

本章围绕端侧人脸表情识别系统的需求分析与架构设计展开说明。首先，结合室内近距离识别场景，明确了系统需要完成视频采集、人脸检测、人脸裁剪、表情分类、结果显示和图像保存等功能，并从准确率、实时性、资源占用和稳定性等方面分析了系统性能需求。

13.0%(789)

其次，本文设计了从数据准备、模型训练、模型评估到树莓派端部署运行的总体架构，并对系统实时识别流程进行了说明。该流程以摄像头图像为输入，依次完成人脸检测、图像预处理、表情分类和结果输出，体现了本文从离线模型训练到端侧实时应用的完整过程。

最后，本章对YuNet人脸检测模型、MobileNetV3表情分类模型和训练优化方法进行了选型说明。下一章将进一步围绕表情分类模型展开，重点介绍数据集构建、预处理方法、模型结构调整和训练策略设计。

4. 第3章表情识别模型构建与训练

AI特征值: 16.6% AI特征字符数 / 章节(部分)字符数: 1025 / 6189

片段指标列表

序号	片段名称	字符数	AI特征		
6	片段1	458	疑似		7.4%
7	片段2	542	显著		8.8%
8	片段3	271	疑似		4.4%
9	片段4	483	显著		7.8%

原文内容

第3章表情识别模型构建与训练

3.1 数据集构建与预处理

第2章已经完成了端侧人脸表情识别系统的需求分析和总体架构设计。本章不再展开树莓派端运行流程，而是围绕表情分类模型本身进行说明，重点介绍训练数据整理、数据集划分、图像预处理、MobileNetV3分类网络调整以及训练策略设计等内容。

在人脸表情识别任务中，训练数据的质量会直接影响模型效果。不同表情类别之间存在一定相似性，例如恐惧、悲伤和厌恶在局部表情特征上容易混淆；同一类表情在不同个体、光照和姿态下也会呈现较大差异。因此，本文在原始FER2013数据基础上进行了样本扩充、合并、清洗和重新划分，并统一整理为CSV格式，作为后续模型训练的输入数据。

3.1.1 数据集构建与扩展

本文表情分类任务以FER2013数据集为基础[4]，并在此基础上补充扩展样本。识别类别共包括七类基本表情，分别为愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中

性，对应类别编号为0至6。

原始FER2013数据集中，愤怒类3110张、厌恶类248张、恐惧类819张、高兴类9355张、悲伤类4370张、惊讶类4462张、中性类12905张，总计35269张。可以看出，原始数据中不同类别样本数量差异较大，其中厌恶类和恐惧类样本数量明显偏少。若直接使用该数据进行训练，模型容易偏向样本数量较多的类别。

为缓解这一问题，本文在原始数据基础上补充了扩展样本，并对数据进行统一整理。扩充后数据集总量达到410246张。为了直观展示扩充前后的变化，本文绘制了原始FER2013数据集与扩充后数据集的类别数量对比图，如图3-1所示。

图 3-1 原始FER2013与扩充后数据集类别数量对比

由图3-1可以看出，扩充后各类样本数量均有所增加。其中，原始数据中较少的厌恶类和恐惧类分别增加至17128张和16954张，数据稀缺问题得到一定缓解。与此同时，高兴类和中性类仍然是样本数量较多的类别，说明扩充后的数据集仍存在类别不均衡现象。因此，后续训练中仍需要引入类别均衡策略，避免模型过度偏向多数类别。

7.4%(458)

3.1.2 CSV数据合并与数据集划分

本文训练过程中没有直接采用文件夹路径读取方式，而是将原始样本和扩展样本统一转换为CSV格式进行管理。每条CSV记录主要包括类别标签和图像像素信息，部分扩展样本也可通过图像路径字段读取。这样处理后，不同来源的数据可以通过统一的数据加载脚本读取，减少了训练阶段对样本来源的额外判断。

在数据整理阶段，本文首先将原始训练、验证和测试数据与扩展样本进行合并，生成统一的数据文件。随后，对样本进行随机打乱，并在各类别内部进行划分，最终得到train.csv、val.csv、test.csv和unlabeled.csv四个文件。其中，训练集用于基础监督训练，验证集用于训练过程中的模型选择，测试集用于最终模型评价，无标签集则用于后续伪标签生成。

本文采用的划分方式为：先在每个类别内部预留约20%的样本作为无标签集；剩余有标签样本再按照8:1:1比例划分为训练集、验证集和测试集。对于无法完全整除的少量样本，统一并入无标签集。这样可以使训练集、验证集和测试集在各类别上保持相近比例，同时为后续伪标签训练预留数据。

数据集划分结果如表3-1所示。

表 3-1 扩充后数据集划分结果

数据集样本数量主要用途

训练集 262536 用于基础监督训练

验证集 32817 用于训练过程中的模型选择

测试集 32817 用于最终模型性能评价

无标签集 82076 用于伪标签生成与辅助训练

合计 410246 —

各子集类别分布如表3-2所示。

表 3-2 划分后各子集类别分布

表情类别训练集验证集测试集无标签集

愤怒 18208 2276 2276 5692

厌恶 10960 1370 1370 3428

恐惧 10848 1356 1356 3394

高兴 93120 11640 11640 29106

悲伤 29192 3649 3649 9125

惊讶 22608 2826 2826 7072

中性 77600 9700 9700 24259

合计 262536 32817 32817 82076

从表3-2可以看出，训练集、验证集和测试集在类别比例上基本保持一致，这有利于保证训练过程和评估过程的数据分布相近。无标签集虽然在CSV文件中保留原始类别字段，但在伪标签生成阶段并不直接使用该字段作为监督标签，而是由训练后的模型重新预测并筛选高置信度样本。

3.1.3 数据预处理与增强策略

由于MobileNetV3模型要求固定尺寸输入，本文在训练前需要对CSV中的图像数据进行还原和预处理。对于FER2013格式样本，训练脚本先将像素序列还原为48×48灰度图，再转换为三通道图像，以适配基于ImageNet预训练权重的MobileNetV3模型。随后，所有输入图像统一调整为224×224像素。

在归一化处理方面，本文采用ImageNet数据集常用的均值和标准差进行标准化处理。这样设置主要是为了与预训练模型的输入分布保持一致，减少从ImageNet预训练模型迁移到表情识别任务时的输入差异。

训练阶段采用适度数据增强，以提高模型对人脸姿态变化、轻微旋转和局部遮挡的适应能力。具体包括随机水平翻转、随机旋转、随机裁剪缩放和随机擦除等操作。其中，随机水平翻转用于模拟左右姿态变化；随机旋转用于模拟头部轻微偏转；随机裁剪缩放用于增强模型对人脸边界变化的容忍度；随机擦除则用于模拟局部遮挡情况。

验证集和测试集不使用随机增强，只进行尺寸调整和归一化处理。这样可以保证评估阶段输入稳定，避免随机增强对评价结果造成干扰。通过上述预处理方式，训练阶段能够提供更多样的输入变化，而评估阶段则保持统一的测试条件。

3.1.4 类别不平衡处理

虽然本文对数据集进行了扩充，但从图3-1和表3-2可以看出，各类别之间仍然存在数量差异。高兴类和中性类样本数量较多，厌恶类和恐惧类相对较少。如果直接采用普通交叉熵损失进行训练，模型可能更容易学习到多数类别特征，而对少数

8.8%(542)

类别表情识别不足。

针对这一问题，本文在训练阶段引入类别均衡策略。训练脚本会统计训练集中各类别样本数量，并根据类别分布计算类别权重。样本数量较少的类别在损失函数中获得相对较高权重，样本数量较多的类别权重相对降低，从而减弱类别数量差异对模型训练的影响。

此外，本文在基础训练阶段还设置了每类样本数量上限，用于控制单轮训练中多数类样本的占比。这样可以在保证训练样本规模的同时，减少多数类样本对训练过程的过度主导。类别均衡策略并不能完全消除类别间混淆，但可以提升训练过程对少数类别的关注度。

4. 4%(271)

3.2 模型架构设计

3.2.1 MobileNetV3骨干网络选型

本文表情分类模型采用MobileNetV3作为骨干网络。MobileNetV3是一种面向移动端和资源受限设备设计的轻量化卷积神经网络，在模型规模、计算量和特征表达能力之间具有较好的平衡。考虑到本文系统最终需要服务于树莓派端实时识别，分类模型不能过大，因此选择MobileNetV3作为基础网络结构。

在具体实现中，本文采用MobileNetV3-large版本。相比MobileNetV3-small，large版本具有更强的特征提取能力，适合在PC端训练后再进行模型转换和端侧部署。由于人脸表情识别任务需要区分眼部、嘴部和面部纹理等细节变化，模型需要具备一定的表达能力。MobileNetV3-large能够在保持相对轻量的同时提供较好的分类基础。

本文使用ImageNet预训练权重初始化模型参数，再根据表情分类任务对输出层进行调整。这样可以利用预训练模型已有的图像特征提取能力，减少从零开始训练带来的不稳定性。

3.2.2 分类层调整与输出设计

原始MobileNetV3模型面向ImageNet分类任务，输出类别数量与本文任务不一致。因此，本文保留MobileNetV3的主干特征提取部分，将最后的全连接分类层替换为七分类输出层。调整后的模型输出维度为7，对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情。

模型输入为224×224的三通道人脸图像。输入图像经过卷积层、倒残差结构和特征聚合后，最终由分类层输出七个类别的logits。训练时，logits用于计算交叉熵损失；推理时，通过Softmax得到各类别预测概率，并选择概率最大的类别作为预测结果。

这种调整方式没有改变MobileNetV3的主体结构，只对任务相关的输出层进行替换，符合迁移学习的基本思路。对于本文任务而言，该方法能够在较小改动下将通用图像分类模型迁移到人脸表情分类任务中。

3.2.3 模型评价指标

为了评价表情分类模型的训练效果，本文主要采用准确率和宏平均F1值作为评价指标。

准确率用于衡量所有样本中被正确分类的比例，可以直观反映模型整体识别效果。但在类别分布不均衡的情况下，仅使用准确率可能会掩盖少数类识别不足的问题。例如，如果模型对样本数量较多的高兴类和中性类识别较好，即使对厌恶类或恐惧类识别一般，总体准确率仍可能较高。

因此，本文同时采用宏平均F1值作为模型选择指标。宏平均F1值会分别计算各类别的F1值，再对各类别结果取平均，更适合评价类别不均衡情况下模型对各类表情的综合识别能力。训练过程中，本文以验证集宏平均F1值作为最优模型保存依据，而不是只依据总体准确率进行模型选择。这样能够使模型在整体识别效果和各类别均衡表现之间取得更合理的平衡。

3.3 训练策略设计

本节主要说明表情分类模型的训练方案设计，重点介绍基础监督训练、伪标签辅助训练和多阶段训练流程。需要说明的是，本节只说明训练思路和参数设置，不展开具体训练结果分析。各阶段训练过程、伪标签生成结果和最终实验结果将在第4章中进一步讨论。

3.3.1 基础监督训练方案

基础监督训练阶段只使用真实标注样本，包括训练集、验证集和测试集中的有标签数据。其中，训练集用于参数更新，验证集用于模型选择，测试集用于后续性能评价。该阶段的目标是先获得一个具有基本表情分类能力的初始模型，为后续伪标签生成提供基础。

训练时，模型输入为经过增强和归一化处理的人脸图像，标签为七类表情编号。损失函数采用带类别权重的交叉熵损失，并结合标签平滑减少模型过度自信。优化器采用AdamW，学习率使用预热和余弦衰减策略。训练过程中根据验证集宏平均F1值保存最优模型，避免只依据训练损失下降来判断模型优劣。

基础监督训练阶段不引入无标签样本，主要用于建立可靠的初始分类器。该阶段训练得到的模型将作为后续伪标签生成的教师模型。

3.3.2 伪标签辅助训练方案

在基础监督训练完成后，本文使用训练得到的模型对无标签集进行预测，并根据预测置信度筛选伪标签样本。伪标签生成时，模型对无标签样本输出七类概率，程序保留置信度达到设定阈值的样本，并将预测类别作为伪标签写入新的CSV文件。

第一轮伪标签生成使用Stage 1模型作为教师模型，筛选阈值设为0.85，并限制每类最多保留一定数量的样本，以避免某些类别伪标签过多。第二轮伪标签生成使用后续训练得到的模型重新预测无标签样本，阈值提高到0.90，以提升伪标签质量。

需要说明的是，伪标签并不等同于真实人工标注，其中可能包含预测错误样本。因此，在后续训练中，本文没有将伪标签作为核心创新点，而是将其作为辅助训练手段。通过置信度筛选、类别数量限制和分阶段引入，可以在一定程度上减少错误伪标签对模型训练的影响。

3.3.3 多阶段训练流程设计

本文训练过程分为三个阶段。各阶段训练数据和模型来源如表3-3所示。

表 3-3 多阶段训练流程设计

阶段训练数据初始化模型主要目标输出模型

Stage 1 有标签训练集 ImageNet预训练模型建立基础表情分类模型 Stage 1最优模型

Stage 2 有标签训练集 + 第一轮伪标签数据 Stage 1最优模型利用伪标签扩展监督信息 Stage 2最优模型

Stage 3 有标签训练集 + 第二轮伪标签数据 Stage 2最优模型更新伪标签质量并继续优化最终分类模型

在Stage 1中，模型只使用真实标注样本进行训练，获得初始分类能力。在Stage 2中，先利用Stage 1模型对无标签集生成第一轮伪标签，再将筛选后的伪标签样本与真实标注样本共同用于训练。在Stage 3中，使用Stage 2模型重新生成伪标签，并继续进行联合训练。

这种多阶段训练流程的目的，是逐步提高无标签样本的利用效率。相比一次性加入全部无标签样本，分阶段筛选伪标签可以使模型先从较可靠的样本开始学习，再逐步更新训练数据。该流程仍属于训练优化策略，不改变MobileNetV3主体结构。

3.3.4 训练参数设置

为了保证训练过程具有较好的稳定性和一致性，本文在各训练阶段采用统一的基础训练参数。

本文各阶段训练的主要参数保持一致：输入尺寸为 224×224 ，batch size设为128，最大训练轮数为200，初始学习率为 $5e-4$ ，最低学习率为 $1e-6$ ，优化器采用AdamW，权重衰减设为 $1e-4$ ，标签平滑系数设为0.04，类别均衡参数beta设为0.995，早停轮数设为20，随机种子设为42。Stage 2和Stage 3在此基础上引入伪标签样本，伪标签样本按置信度筛选后参与后续训练。

在训练稳定性方面，本文主要采用以下处理方式。首先，学习率使用线性预热和余弦衰减，避免训练初期学习率过大造成参数波动。其次，采用AdamW优化器和权重衰减，以降低模型过拟合风险。再次，启用自动混合精度训练，以减少显存占用并提高训练效率。最后，使用梯度裁剪限制梯度异常波动，增强训练过程稳定性。

在模型保存方面，训练过程以验证集宏平均F1值作为最优模型判断依据。当验证集宏平均F1值提升时，保存当前模型；如果连续多轮没有提升，则触发早停机制

7.8%(483)

。该策略可以避免训练后期过拟合，同时保证最终保存模型具有较好的综合分类能力。

3.4 本章小结

本章围绕表情分类模型的构建与训练方案进行了说明。首先，介绍了数据集来源、扩充过程和CSV格式整理方式，并将扩充后数据集划分为训练集、验证集、测试集和无标签集，为后续监督训练和伪标签辅助训练提供数据基础。

其次，本章说明了图像预处理和数据增强方法。训练阶段采用随机翻转、随机旋转、随机裁剪缩放和随机擦除等操作，以提升模型对姿态变化和局部遮挡的适应能力；验证集和测试集只进行尺寸调整和归一化处理，以保证评价过程稳定。

最后，本章完成了MobileNetV3表情分类模型设计和训练策略说明。模型以MobileNetV3-large为骨干网络，将分类层调整为七类输出，并采用类别均衡、标签平滑、学习率调度、早停和伪标签辅助训练等方法提高训练稳定性。下一章将基于本章设计的训练流程，对不同阶段的训练结果和最终模型表现进行分析。

5. 第4章模型训练优化

AI特征值：**9.7%**

AI特征字符数 / 章节(部分)字符数：382 / 3955

片段指标列表

序号	片段名称	字符数	AI特征		
10	片段1	367	疑似		9.3%
11	片段2	382	显著		9.7%

原文内容

第4章模型训练优化

4.1 训练优化思路与总体流程

在第3章完成数据集构建、模型结构调整和训练策略设计后，本章进一步对模型训练过程和模型训练结果进行分析。本文的训练目标并不是单纯追求复杂算法结构，而是在MobileNetV3轻量化模型基础上，通过数据扩充、类别均衡、伪标签辅助训练和多阶段优化等方式，获得能够支撑后续树莓派端部署的表情分类模型。

从整体训练流程来看，本文将模型训练分为三个阶段。第一阶段为基础监督训练，仅使用有标签训练集对MobileNetV3-large模型进行训练，目的是获得一个具备基本表情分类能力的初始模型。第二阶段在第一阶段模型基础上生成第一轮伪标签，并将有标签样本与伪标签样本联合用于训练，使模型能够利用无标签数据中的有效信息。第三阶段使用第二阶段模型重新生成伪标签，并继续进行联合训练，以进

一步稳定模型分类能力。

其中，伪标签辅助训练的作用是补充可利用样本信息，提高无标签数据的利用效率，但模型最终效果仍需通过验证集、测试集和混淆矩阵等结果进行综合评价。模型最终效果仍需通过验证集、测试集和混淆矩阵等结果进行综合评价。为了避免与第3章内容重复，本章不再展开训练参数和数据增强细节，而重点分析多阶段训练结果、最终模型评价结果以及各类别识别表现。

4.2 多阶段训练结果对比

为比较不同训练阶段对模型性能的影响，本文分别对Stage 1、Stage 2和Stage 3阶段保存的最优模型进行统一评估。评估时采用验证集和测试集作为评价数据，并统计准确率和宏平均F1值。其中，准确率反映模型总体分类正确比例，宏平均F1值能够更好地反映类别不均衡场景下模型对各类表情的综合识别能力。

多阶段训练结果如表4-1所示。

表4-1 多阶段模型评价结果对比

训练阶段	训练数据	Val Acc/%	Val Macro-F1/%	Test Acc/%	Test Macro-F1/%
Stage 1	有标签训练集	71.57	64.33	71.85	64.57
Stage 2	有标签训练集 + 第一轮伪标签数据	90.58	87.48	90.82	87.56
Stage 3	有标签训练集 + 第二轮伪标签数据	90.55	87.32	90.77	87.43

由表4-1可以看出，Stage 1阶段仅使用有标签样本训练，模型已经能够完成基本七类表情分类任务，但整体效果仍有较大提升空间。该阶段测试集准确率为71.85%，宏平均F1值为64.57%，说明基础模型虽然具备初步识别能力，但在类别分布不均衡和复杂表情样本上仍存在不足。

Stage 2阶段引入第一轮伪标签数据后，模型性能出现明显提升。测试集准确率由Stage 1的71.85%提升至90.82%，宏平均F1值由64.57%提升至87.56%。这说明在基础监督模型已经具有一定判别能力的前提下，引入高置信度伪标签样本能够有效补充训练信息，使模型学习到更加充分的表情特征。

Stage 3阶段使用Stage 2模型重新生成伪标签后继续训练，其测试集准确率为90.77%，宏平均F1值为87.43%，与Stage 2结果基本接近。该结果说明，经过Stage 2训练后，模型性能已经进入相对稳定区间，第二轮伪标签更新没有带来明显增益，但也没有造成模型性能明显下降。因此，后续系统部署采用Stage 3模型作为最终分类模型，主要是因为该模型来自完整的多阶段训练流程，能够较好地衔接模型训练、评估与部署过程。

从整体结果来看，伪标签辅助训练的主要提升集中在Stage 1到Stage 2之间。Stage 3更多体现为训练流程的进一步完善和模型效果的稳定保持。

4.3 最终模型总体评价

在完成多阶段训练后，本文选取Stage 3阶段输出的最终模型进行验证集和测试

9.3%(367)

集评价。最终模型的总体评价结果如表4-2所示。

表4-2 最终模型总体评价结果

数据集 Loss Accuracy/% Macro-F1/%

验证集 0.4054 90.55 87.32

测试集 0.3970 90.77 87.43

由表4-2可以看出，最终模型在验证集和测试集上的结果较为接近。其中，验证集准确率为90.55%，宏平均F1值为87.32%；测试集准确率为90.77%，宏平均F1值为87.43%。测试集结果略高于验证集，但二者差异很小，说明模型在当前数据划分下没有出现明显过拟合，具备较稳定的泛化表现。

从评价指标角度看，准确率超过90%说明模型能够较好地完成整体七类表情分类任务；宏平均F1值达到87%左右，说明模型并不是单纯依靠高频类别获得较高准确率，而是在各类别上也保持了较均衡的识别能力。考虑到本文最终目标是将模型部署到树莓派端进行实时识别，该模型在准确率、类别均衡表现和轻量化结构之间取得了较好的平衡，能够作为后续端侧系统部署的分类模型基础。

需要注意的是，本文的模型训练目标并不是在公开数据集上追求最高指标，而是获得适合端侧部署的稳定分类模型。因此，在最终模型选择时，除了关注测试集准确率外，还需要结合宏平均F1值、混淆矩阵和各类别表现进行综合判断。

4.4 混淆矩阵与类别识别分析

为了进一步分析最终模型在不同表情类别上的识别情况，本文绘制了测试集混淆矩阵，如图4-1所示。图中纵轴表示真实类别，横轴表示预测类别，类别编号0至6分别对应愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性。

图4-1 测试集混淆矩阵

从图4-1可以看出，大部分样本集中在混淆矩阵的主对角线上，说明模型对各类别均具备较好的识别能力。其中，高兴类和中性类样本数量较多，且主对角线位置数值明显较高，表明模型对这两类表情具有较强的识别能力。高兴类共有11121个样本被正确识别，中性类共有9023个样本被正确识别。

为了进一步量化不同类别的表现，本文统计了测试集上各类别的精确率、召回率和F1值，结果如表4-3所示。

表4-3 测试集各类别识别结果

类别编号表情类别 Precision/% Recall/% F1/%

0 愤怒 87.62 87.08 87.35

1 厌恶 88.04 80.07 83.87

2 恐惧 85.27 78.54 81.77

3 高兴 97.05 95.54 96.29

4 悲伤 83.76 81.72 82.73

5 惊讶 90.05 89.07 89.56

由表4-3可以看出，高兴类的识别效果最好，其Precision、Recall和F1值分别为97.05%、95.54%和96.29%。这说明高兴表情的视觉特征较明显，例如嘴角上扬、面部肌肉变化较突出，模型较容易学习到该类表情的判别特征。中性类的F1值为90.47%，召回率达到93.02%，说明模型对中性表情也具有较好的识别能力。惊讶类F1值为89.56%，整体表现同样较稳定。

相比之下，恐惧类、悲伤类和厌恶类的F1值相对较低。其中，恐惧类F1值为81.77%，是七类表情中最低的一类；悲伤类F1值为82.73%，厌恶类F1值为83.87%。这说明这些类别在视觉表现上更容易与其他类别产生混淆。例如，恐惧表情在面部局部特征上可能与惊讶、悲伤或中性存在相似性；悲伤表情在表情强度较弱时容易接近中性；厌恶类样本数量相对较少，且面部表现差异较大，因此识别难度也较高。

9.7%(382)

结合混淆矩阵进一步观察可以发现，部分样本容易被预测为中性类。例如，真实悲伤类中有446个样本被误判为中性，真实高兴类中有353个样本被误判为中性，真实厌恶类中有150个样本被误判为中性，真实愤怒类中也有110个样本被误判为中性。这说明在表情强度较弱或面部特征不明显时，模型倾向于将样本归入中性类别。另一方面，中性类也存在被误判为悲伤和高兴的情况，其中有300个中性样本被预测为悲伤，163个中性样本被预测为高兴。这表明中性类别与弱表情类别之间仍存在一定边界模糊问题。

总体来看，最终模型在七类表情上均取得了可用的识别效果，尤其在高兴、中性和惊讶等类别上表现较好；但对于恐惧、悲伤和厌恶等类别，仍存在一定程度的混淆。该结果符合人脸表情识别任务本身的特点，也为后续模型改进提供了方向。后续可以通过进一步提高少数类样本质量、增强弱表情样本标注一致性、引入更加精细的人脸区域特征或注意力机制等方式，改善易混类别的识别效果。

4.5 本章小结

本章围绕表情分类模型的训练优化过程和实验结果进行了分析。首先，本文对Stage 1、Stage 2和Stage 3三个阶段的训练结果进行了对比。结果表明，仅使用有标签数据进行基础监督训练时，模型能够完成初步分类任务，但整体性能有限；在Stage 2中引入高置信度伪标签后，模型准确率和宏平均F1值均得到明显提升；Stage 3在第二轮伪标签更新后与Stage 2结果基本接近，说明模型性能已经趋于稳定。

其次，本文对最终Stage 3模型进行了总体评价。最终模型在验证集上的准确率为90.55%，宏平均F1值为87.32%；在测试集上的准确率为90.77%，宏平均F1值为87.43%。验证集和测试集结果差异较小，说明模型具备较稳定的分类能力。

最后，通过测试集混淆矩阵和各类别指标分析可以看出，模型对高兴、中性和惊讶等类别识别效果较好，但对恐惧、悲伤和厌恶等类别仍存在一定混淆。总体而

言，最终表情分类模型已经能够满足后续树莓派端部署和实时识别系统测试的基本要求。下一章将在本章训练结果基础上，进一步介绍模型转换、树莓派端部署和系统运行测试过程。

6. 第5章系统实现与运行测试

AI特征值: 10.7%

AI特征字符数 / 章节(部分)字符数: 670 / 6273

片段指标列表

序号	片段名称	字符数	AI特征		
12	片段1	336	疑似		5.4%
13	片段2	670	显著		10.7%

原文内容

第5章系统实现与运行测试

5.1 系统实现与运行流程

前几章已经完成了端侧人脸表情识别系统的需求分析、模型构建和训练优化。第4章实验结果表明，最终Stage 3模型在测试集上取得了较稳定的识别效果，因此本章进一步围绕模型转换、树莓派端部署和实际运行测试展开说明。

本文系统最终目标不是停留在PC端离线分类实验，而是将训练完成的表情分类模型部署到树莓派端，结合摄像头输入和YuNet人脸检测模型，实现实时人脸表情识别。系统运行时，摄像头采集视频画面，程序调用YuNet检测人脸区域和关键点，然后根据检测框裁剪人脸图像，将其输入TFLite表情分类模型，最后在显示界面中绘制人脸框、关键点、表情类别、置信度、FPS和耗时信息。

在整体实现上，PC端主要承担模型训练、模型导出和格式转换任务；树莓派端主要承担视频采集、人脸检测、表情分类、结果显示和图像保存任务。通过这种方式，本文完成了从模型训练、模型转换到树莓派端运行验证的完整闭环。本章重点通过模型导出文件、端侧推理参数和实际运行截图说明系统实现过程。

5.2 软件环境与硬件平台配置

5.2.1 硬件平台配置

本文系统涉及PC端训练转换平台和树莓派端部署平台。PC端用于数据处理、模型训练、模型评估和模型格式转换；树莓派端用于部署TFLite模型并完成实时识别测试。

硬件平台配置如表5-1所示。

表5-1 硬件平台配置

5.4%(336)

平台配置项具体信息

PC端训练与转换平台处理器 Intel Core i7-10870H @ 2.20GHz

PC端训练与转换平台内存 32.0 GB

PC端训练与转换平台 GPU NVIDIA RTX3060

端侧部署平台开发板 Raspberry Pi 5

端侧部署平台摄像头 H052-8614

端侧部署平台摄像头采集分辨率 640×480

PC端硬件主要用于完成多阶段模型训练和模型转换任务。树莓派5则作为最终部署平台，用于验证模型在端侧环境下的实际运行效果。由于树莓派平台算力和内存资源有限，部署阶段需要控制模型大小、推理频率和单帧处理负载。

5.2.2 软件环境与工具

本文模型训练阶段主要使用PyTorch完成MobileNetV3表情分类模型训练；模型转换阶段使用ONNX、onnxruntime、onnxsim、onnx2tf和TensorFlow Lite等工具；树莓派端使用OpenCV、YuNet和TFLite运行时完成实时推理。模型导出脚本中设置最终导出模型类别数为7，模型名称为MobileNetV3-large，加载的权重文件为best_model_stage3.pth，导出输入尺寸为224，并采用NCHW输入布局。

软件工具及用途如表5-2所示。

表5-2 软件工具与用途

软件或工具主要用途

Python 训练、模型转换和树莓派端推理程序运行环境

PyTorch 加载和训练MobileNetV3表情分类模型

ONNX 作为PyTorch模型到其他框架之间的中间格式

onnxsim 对ONNX模型进行简化

onnxruntime 对导出的ONNX模型进行快速验证

TensorFlow / onnx2tf 将ONNX模型转换为TensorFlow SavedModel

TensorFlow Lite / tflite-runtime 在树莓派端执行TFLite模型推理

OpenCV 摄像头读取、YuNet检测、图像绘制和窗口显示

5.3 模型导出与端侧部署

5.3.1 模型导出与格式转换

第4章确定Stage 3模型作为最终部署模型。该模型最初以PyTorch权重文件形式保存，不能直接在树莓派端高效运行，因此需要将其转换为适合端侧部署的TFLite模型。

本文模型转换过程包括以下步骤。首先，加载训练得到的best_model_stage3.pth权重文件，并构建输出类别数为7的MobileNetV3-large模型。其次，将PyTorch模型导出为ONNX格式。再次，使用ONNX简化工具生成简化后的ONNX模型。随后，将简化后的ONNX模型转换为TensorFlow SavedModel格式。最后

，使用TensorFlow Lite转换器生成TFLite模型。导出脚本中量化方式设置为FP16，验证样本数量设置为200，用于检查转换后模型输出的一致性。

模型转换后得到的主要文件如表5-3所示。

表5-3 模型导出文件

文件类型	大小	作用
model.onnx	ONNX文件 16445 KB	PyTorch导出的ONNX中间模型
model_simplified.onnx	ONNX文件 16459 KB	简化后的ONNX模型
saved_model	文件夹	TensorFlow SavedModel目录
saved_model.pb	PB文件 16570 KB	SavedModel模型文件
model_simplified_float32.tflite	TFLite文件 16451 KB	float32格式

TFLite模型

model_fp16.tflite TFLite文件 8265 KB FP16量化后的TFLite模型

由表5-3可以看出，FP16量化后的 model_fp16.tflite 文件大小约为8265 KB，而float32格式TFLite模型约为16451 KB。FP16量化后模型文件体积约减少一半[14][15]，能够降低树莓派端存储占用，也更适合端侧部署。

5.3.2 模型转换一致性验证

模型格式转换可能导致输出结果发生偏移，因此在部署前需要进行一致性验证。本文使用转换脚本对PyTorch、ONNX和TFLite三种形式的模型输出进行对比，验证样本数量为200。验证报告显示，PyTorch与ONNX的平均最大绝对误差约为，PyTorch与TFLite的平均最大绝对误差约为0.0068，ONNX与TFLite的平均最大绝对误差也约为0.0068；三种模型在验证样本上的预测一致性均为1.0。

模型转换一致性验证结果如表5-4所示。

表5-4 模型转换一致性验证结果

对比项	平均最大绝对误差	平均绝对误差
PyTorch 与 ONNX	2.50×10^{-6}	1.18×10^{-6}
PyTorch 与 TFLite	0.0068	0.0033
ONNX 与 TFLite	0.0068	0.0033

从表5-4可以看出，PyTorch模型与ONNX模型之间误差极小，说明ONNX导出过程基本保持了模型输出一致性。TFLite模型由于采用FP16量化，输出与原PyTorch模型之间存在小幅数值差异，但差异范围较小，预测结果保持一致。因此，本文认为该TFLite模型能够作为树莓派端部署模型使用。

5.4 树莓派端推理程序设计

5.4.1 推理程序总体结构

树莓派端推理程序主要包括摄像头读取、YuNet人脸检测、目标跟踪、人脸区域裁剪、TFLite表情分类、结果绘制和图像保存等部分。程序启动后，首先加载TFLite表情分类模型和YuNet人脸检测模型，然后初始化摄像头并进入实时循环。

树莓派端推理程序加载FP16量化后的model_fp16.tflite作为表情分类模型，并加载face_detection_yunet_2023mar.onnx作为YuNet人脸检测模型。程序中的表情类别标签依次为anger、disgust、fear、happy、sad、surprise和neutral，输入图像尺寸设置为224，并使用与训练阶段一致的ImageNet均值和标准差进行归一化处理。

推理程序主要流程如下：

第一，摄像头读取视频帧。程序通过异步摄像头读取类获取实时图像，避免摄像头读取阻塞主循环。

第二，YuNet检测人脸。程序对缩放后的输入图像执行人脸检测，得到检测框和五点关键点。

第三，目标跟踪与位置更新。对于非检测帧，程序利用上一轮检测结果维持目标状态，减少重复检测开销。

第四，人脸区域裁剪。程序根据检测框扩展人脸区域，并裁剪出分类模型输入图像。

第五，TFLite模型推理。裁剪后的人脸图像经过RGB转换、缩放和归一化后输入TFLite模型，输出七类表情概率。

第六，结果绘制与保存。程序在画面中绘制检测框、关键点、预测类别、置信度、FPS和耗时信息，并异步保存高置信度图像。

5.4.2 树莓派端运行参数

树莓派平台算力有限，如果对每一帧图像都执行完整检测和分类，容易造成画面卡顿。因此，本文在端侧推理程序中设置了多项运行参数，以控制检测和分类负载。

主要运行参数如表5-5所示。

表5-5 树莓派端推理参数设置

参数设置值作用

摄像头分辨率 640×480 获取实时输入画面

处理分辨率 640×480 主程序处理图像尺寸

YuNet检测输入尺寸 256×192 降低人脸检测计算量

检测间隔 detect_every=3 每3帧执行一次完整检测

分类间隔 infer_every=2 避免连续帧重复分类

YuNet置信度阈值 0.7 过滤低置信度检测框

NMS阈值 0.3 去除重复检测框

TFLite线程数 4 提高分类推理效率

表情置信度阈值 0.45 过滤低置信度分类结果

人脸框扩展比例 0.18 保留较完整的人脸区域

最大跟踪人脸数 4 控制画面中最多跟踪目标

单帧最多分类人脸数 1 降低多目标分类负载

目标帧率参数 30 FPS 控制显示刷新节奏

在人脸检测部分，程序将检测输入尺寸设置为 256×192 ，检测结束后再将检测框和关键点映射回原始画面尺寸。这样可以减少YuNet检测阶段的计算量，同时保证画面中检测框位置正确。

在表情分类部分，程序只对主要人脸进行分类推理。虽然程序最多可跟踪4个人脸，但单帧最多只对1个人脸执行表情分类，这一设置能够降低多目标场景下的计算压力。对于毕业设计中的室内单人演示场景，该策略能够在识别功能和实时性之间取得较好的平衡。

5.5 系统运行测试与效果分析

5.5.1 功能测试

为验证系统各模块能否正常运行，本文对摄像头采集、人脸检测、表情分类、结果显示、图像保存和多人场景显示等功能进行了测试。

从功能测试结果看，系统已经能够完成从摄像头输入到识别结果显示的完整流程。YuNet模块能够检测人脸并输出关键点，TFLite分类模型能够输出表情类别和置信度，结果显示模块能够实时叠加运行信息，图像保存模块能够保存部分识别效果较好的画面。

5.5.2 实时性观察

由于本文未单独使用日志系统持续记录每一帧的性能数据，因此本节实时性结果主要来自系统运行界面中显示的FPS、延迟、检测耗时、分类耗时和主循环耗时。测试场景为室内近距离人脸表情识别，显示方式为VNC远程界面。

从运行截图可以观察到，系统在单人场景下FPS通常保持在约47~57之间，部分画面中FPS显示为52.5、53.6、53.3和56.9；在多人场景示例中，FPS约为50.9。YuNet检测耗时多处于约6~14 ms范围内，说明轻量化检测模型和低分辨率检测输入能够有效降低检测开销。表情分类耗时相对检测耗时更高，多数情况下约为18~36 ms，个别画面中达到60 ms以上，说明TFLite表情分类仍是树莓派端推理流程中的主要耗时部分。不同画面中的延迟存在一定波动，主要与摄像头读取、检测与分类推理负载有关。

系统实时性观察结果如表5-6所示。

表5-6 系统实时性观察结果

指标观察结果

摄像头采集分辨率 640×480

目标显示帧率 30 FPS

运行界面FPS 约47~57 FPS

YuNet检测耗时约6~14 ms

表情分类耗时约18~36 ms

测试场景室内近距离单人及少量多人场景

从表5-6可以看出，系统在室内测试环境下能够保持较连续的画面显示，检测框和识别类别能够随人脸状态变化进行更新。YuNet检测耗时较低，主要得益于检测输入尺寸被降低至 256×192 以及间隔检测策略。分类耗时相对检测耗时更高，说明TFLite表情分类仍然是树莓派端主要计算开销之一。

5.5.3 运行效果展示

为了展示系统实际运行效果，本文选取了若干树莓派端运行截图，覆盖愤怒、恐惧、高兴、中性、悲伤等单人表情识别场景，以及少量多人识别场景。运行效果如图5-1和图5-2所示。

图5-1 单人场景下多类别表情识别效果

图5-1展示了单人场景下的多类别表情识别效果。可以看出，系统能够在画面中绘制人脸检测框和五点关键点，并在检测框附近显示目标编号、预测类别和置信度。画面左上角同时显示FPS、延迟、检测耗时、分类耗时和主循环耗时等信息，便于观察系统实时运行状态。

图5-2 多人场景下表情识别效果

图5-2展示了多人场景下的识别效果。当画面中出现两个人脸时，系统能够检测并绘制多个人脸框。由于程序设置了`max_infer_faces=1`，系统优先对主要目标进行表情分类，以降低多人场景下的分类推理负载。因此，当前系统更适合单人近距离实时识别场景，在多人场景下仍需要进一步优化多目标分类策略。

5.5.4 稳定性测试

为验证系统连续运行能力，本文在树莓派端对系统进行了稳定性观察。测试过程中，系统持续执行摄像头读取、人脸检测、表情分类、结果显示和图像保存等操作，观察程序是否出现崩溃、摄像头中断、界面卡死或结果无法更新等异常情况。

测试结果表明，在室内近距离场景下，系统能够较稳定地完成实时采集、检测、分类和显示任务。异步摄像头读取和异步图像保存机制对系统稳定性有一定帮助，使视频读取和图像写入不会明显阻塞主循环。

不过，当前稳定性测试主要基于人工观察，测试时长和场景复杂度仍然有限。后续可以进一步增加长时间运行测试和多场景测试，例如弱光、逆光、多人移动、侧脸和遮挡等情况，以更全面地评价系统稳定性。

5.6 本章小结

本章围绕人脸表情识别系统的实现、模型部署和运行测试进行了说明。首先，本文介绍了PC端与树莓派端的分工关系，PC端主要完成模型训练、导出和转换，树莓派端主要完成摄像头输入、YuNet人脸检测、TFLite表情分类和结果显示。

其次，本文说明了模型从PyTorch权重文件转换为ONNX、TensorFlow SavedModel和TFLite模型的过程。最终部署模型采用FP16量化后的`model_fp16.tflite`，文件大小约为8265 KB，相比float32格式TFLite模型明显减小

10.7%(670)

。模型转换一致性验证结果表明，PyTorch、ONNX和TFLite模型在验证样本上的预测结果保持一致，说明转换后的模型可以用于端侧部署。

最后，本文对树莓派端系统进行了功能测试、实时性观察、运行效果展示和稳定性测试。测试结果表明，系统能够在室内近距离场景下完成摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能，运行画面较为连续，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。总体来看，本文系统已经实现了从模型训练、格式转换到树莓派端实时运行的完整工程闭环。

7. 第6章结论与展望

AI特征值: 14.0%

AI特征字符数 / 章节(部分)字符数: 311 / 2225

片段指标列表

序号	片段名称	字符数	AI特征		
14	片段1	363	疑似		16.3%
15	片段2	311	显著		14.0%

原文内容

第6章 结论与展望

6.1 结论

本毕业设计围绕端侧人脸表情识别系统的设计与实现展开，完成了从数据集构建、模型训练、模型转换到树莓派端实时运行的完整流程。系统以“轻量化人脸检测 + 轻量化表情分类 + 树莓派端部署”为主要思路，前端采用YuNet完成人脸检测与关键点定位，后端采用MobileNetV3-large构建七类表情分类模型，最终在树莓派平台上实现了摄像头采集、人脸检测、表情分类、结果显示和图像保存等功能。

在模型训练方面，本文以FER2013数据集为基础，对样本进行了扩充、合并、清洗和重新划分，并统一转换为CSV格式进行训练。针对表情类别分布不均衡的问题，训练过程中采用了数据增强、类别均衡、标签平滑和伪标签辅助训练等方法。实验结果表明，最终Stage 3模型在测试集上取得了90.77%的准确率和87.43%的宏平均F1值，能够较好地完成愤怒、厌恶、恐惧、高兴、悲伤、惊讶和中性七类表情识别任务。其中，高兴、中性和惊讶等类别识别效果较好，恐惧、悲伤和厌恶等类别仍存在一定混淆。

在系统实现方面，本文完成了PyTorch模型到ONNX、TensorFlow SavedModel和TensorFlow Lite模型的转换，并采用FP16量化方式生成适合端侧运行的TFLite模型。量化后的model_fp16.tflite 文件体积约为8265 KB，相比float32格式模型明显

减小。模型转换后，本文还对PyTorch、ONNX和TFLite模型输出进行一致性验证，保证转换后的模型能够用于端侧部署。

在树莓派端部署方面，系统集成了YuNet人脸检测、TFLite表情分类、实时画面显示和异步图像保存等功能。为了降低端侧计算负载，系统设置了较低的人脸检测输入分辨率，并采用间隔检测、间隔分类、限制单帧分类人脸数和异步读取等方式提高运行连续性。实际运行结果表明，系统在室内近距离场景下能够较稳定地完成实时识别任务，运行画面较为连续，检测框、关键点、表情类别和置信度能够正常显示，连续运行过程中未观察到程序崩溃、摄像头中断或界面卡死等异常情况。

总体来看，本文完成了一个可运行的端侧人脸表情识别原型系统。该系统不是单纯停留在离线模型训练层面，而是实现了从模型训练到树莓派端部署运行的工程闭环。

6.2 展望

虽然本文系统已经完成基本功能并能够在树莓派端运行，但仍存在一些不足，需要在后续工作中进一步改进。

第一，模型对部分类别的识别能力仍有提升空间。从实验结果可以看出，恐惧、悲伤和厌恶等类别的F1值相对较低，且容易与中性、惊讶等类别发生混淆。后续可以进一步补充少数类和弱表情样本，提高数据标注一致性，也可以尝试引入更精细的注意力机制或面部局部区域特征，以改善易混类别的识别效果。

第二，系统当前更适合室内近距离单人场景。在多人场景下，程序虽然能够检测多个人脸，但为了保证实时性，当前主要对主要目标进行表情分类，无法同时对所有人脸进行高频分类。后续可以结合更高效的跟踪策略和批量推理方式，提升多人场景下的识别能力。

第三，端侧性能测试还可以进一步完善。本文实时性结果主要来自运行界面观察，虽然能够反映系统基本运行状态，但还缺少长时间自动化日志统计。后续可以在程序中加入性能记录模块，持续统计FPS、检测耗时、分类耗时、CPU占用率和内存占用情况，从而更准确地评估系统在不同场景下的运行表现。

第四，系统还可以进一步优化部署方式。当前模型采用FP16 TFLite格式运行，后续可以继续尝试INT8量化、模型剪枝或硬件加速推理方式，以进一步降低模型体积和推理耗时。同时，也可以针对树莓派端运行环境优化摄像头读取、显示刷新和图像保存逻辑，提高系统整体稳定性和响应速度。

综上，本文系统已经完成端侧人脸表情识别的基本设计与实现，后续可从数据质量、模型结构、多目标识别和端侧性能优化等方面继续改进，使系统在更复杂的实际场景中具有更好的适应能力。

致谢

时光匆匆，大学阶段的学习生活即将结束。回顾本次毕业设计从选题、资料查

阅、系统设计、模型训练到最终部署测试的整个过程，我在专业知识、实践能力和问题解决能力等方面都得到了较大的锻炼和提升。在此，我谨向所有给予我指导、帮助和支持的老师、同学以及家人表示诚挚的感谢。

首先，衷心感谢我的指导教师康莉莉在毕业设计全过程中给予的耐心指导。从课题方向确定、系统方案设计，到毕业设计结构调整、实验结果分析和格式规范修改，老师都提出了许多宝贵意见，使我能够不断发现问题并及时改进。老师严谨认真的治学态度和负责细致的指导方式，使我受益匪浅。

其次，感谢学院各位老师在本科阶段对我的培养。通过四年的课程学习和实践训练，我逐步掌握了人工智能、计算机视觉、程序设计和系统开发等方面的基础知识，也为本次毕业设计的完成奠定了重要基础。

同时，感谢同学和朋友们在毕业设计过程中给予的帮助与鼓励。在系统调试、模型训练、资料整理和毕业设计修改过程中，他们的交流与建议帮助我更好地理清思路，也让我在遇到困难时能够保持信心。

最后，感谢我的家人一直以来的理解、支持和陪伴。正是他们在学习和生活上的鼓励，使我能够顺利完成本科阶段的学习任务。

本次毕业设计仍有许多不足之处，但它也是我本科阶段一次重要的综合实践。今后我将继续保持学习和探索的态度，不断提升自己的专业能力和实践能力。

附录

14.0%(311)

说明:

- 1、支持中、英文内容检测；
- 2、AI特征值=AI特征字符数/总字符数；
- 3、红色代表AI特征显著部分，计入AI特征字符数；
- 4、棕色代表AI特征疑似部分，未计入AI特征字符数；
- 5、检测结果仅供参考，最终判定是否存在学术不端行为时，需结合人工复核、机构审查以及具体学术政策的综合应用进行审慎判断。



关注微信公众号